Libraries and Tools

COURSE EXERCISES

Exercise 1 : A Guild Roster 2

Exercise 2 : A Listening Log Tool 5

Exercise 3 : Checking a Music Library 8

Exercise 4 : Scraping a Page Into a Database 10

Exercise 5 : SQLAlchemy 16

Exercise 6 : Listening Habits with NumPy 20

Exercise 7 : Pandas 23

APPENDIX

Exercise 9 : Running Other Programs 29

Exercise 1 : A Guild Roster

Build a roster whose members share character state but supply different attacks. Health, mana and rage are simplified teaching values, not a model of the game. Start from `exercises/starters/01-inheritance/` and run from `libraries/`. The final solution is in `solutions/solution01-inheritance/`.

Step 1 : Define a character

Create `characters.py` with a plain `Character` class. Store `name` and an initial `health` of 100. Add `attack() -> str` returning a description. Construct a character and print its attributes and attack.

> ***Hint :*** *The slides Base Classes and Subclasses and Inherited Members show the shared attributes and method on* `Character`.

Step 2 : Add specialisations

Create `Mage(Character)` with `mana`, and `Warrior(Character)` with `rage`. Call `super().__init__(name)` in each initialiser. Override `attack` so a mage casts Frostbolt and a warrior uses Heroic Strike. Add `Priest(Character)` with a Smite attack; it can inherit the base initialiser unchanged.

> ***Hint :*** *The slides Constructor Chaining and Overriding Methods show* `super` *and method replacement. Demo* `polymorphism` *includes all three subclasses.*

Step 3 : Give objects a display

Implement `__str__` and `__repr__` on `Character`. Print one mage directly, then a list containing it. Check which method each display uses. Also compare `isinstance(mage, Character)` with `type(mage) is Character`.

> ***Hint :*** *The slides str and repr and isinstance and issubclass show these operations.*

Step 4 : Build the roster

Create `guild.py`. A `Guild` holds a `list[Character]`. Give it `recruit` to add a member and `report` to return the members' attack descriptions. The report must call `member.attack()` without checking a member's subclass. Recruit Jaina as a mage, Varian as a warrior and Anduin as a priest, then print the report.

> ***Hint :*** *The slide Polymorphism shows one loop calling each object's implementation. Composition shows one object holding other objects.*

Step 5 : Enforce the interface

Make `Character` an abstract base class with abstract `attack`. Keep its initialiser and display methods concrete. Try constructing `Character` and an unfinished subclass, then remove those failing calls. Confirm that all three complete subclasses still work with the roster.

> ***Hint :*** *The slides Abstract Base Classes and Enforcing the Contract show `ABC`, `abstractmethod` and the construction error for an incomplete subclass.*

Step 6 : Add belongings

Add `Mount` and `Inventory`. A character has an inventory and may have a mount; it does not inherit from either class. Give Jaina a Gryphon and a Healing Potion. Print the mount name and inventory contents through the character attributes.

> ***Hint :*** *The slide Composition and demo `composition` show collaborator attributes.*

If time permits

Count mages with `isinstance`. Add another concrete character class without changing `Guild.report`. Give the inventory a method to add an item instead of appending to its list from the caller.

> ***Hint :*** *The slides isinstance and issubclass and Polymorphism show how these changes keep the roster independent of specific subclasses.*

Exercise 2 : A Listening Log Tool

Turn a fixed listening report into `logtool.py`, with `count` and `errors` subcommands. `data/listening.log` is fictional personal history. Each line has an ISO date, time, level, action and track title. Titles can contain spaces. `count` reports successful `INFO` plays and skips per title. `errors` selects lines at one level.

Start from `exercises/starters/02-command-line-programs/`. Run from `libraries/`. The complete solution is in `solutions/solution02-command-line-programs/`. Use invented data, not files from work.

Step 1 : Accept a file

Run `solutions/solution02-command-line-programs/logreport_start.py` to see the fixed report. Give your own script an `ArgumentParser` with a positional `logfile` argument converted to `Path`. Read that path instead of a constant. Use `line.split(maxsplit=4)` to retain a title containing spaces.

> ***Hint :*** *The slides ArgumentParser and Path Arguments show positional arguments and `type=Path`. The fundamentals string demos show `split`; `maxsplit` limits how many splits are made. The starter supplies the line-reading code.*

Step 2 : Add subcommands

Register `count` and `errors` with `add_subparsers`. Attach each handler with `set_defaults`. Both receive a logfile path. Give `errors` a `--level` option with default `ERROR`.

Implement `count` with a dictionary or `Counter` keyed by action and title. Print `PLAY` and `SKIP` counts for each title, ignoring other actions and non-INFO lines. Implement `errors` by printing matching lines.

> ***Hint :*** *The slide Subcommands and demo `subcommands` show handler registration. The starter supplies the counting function, so the required work is the CLI.*

Step 3 : Filter dates

Add `--since` to both commands. Accept an ISO date such as `2026-10-13` or a fixed-width timestamp such as `2026-10-12 09:00:00`. Keep lines whose first 19 characters are at least that value. ISO dates in this format sort in date order as strings. Keep the unfiltered lines when the option is absent.

> ***Hint :*** *The slide Options and Flags shows optional values. The fundamentals string and comparison slides show slices and `>=`.*

Step 4 : Report statuses

Return 0 for a successful count or no matching errors, 1 when `errors` finds lines, and 2 when the file cannot be read. Print file errors on stderr. Use `raise SystemExit(main())` at the entry point. With no verb, show help.

Run these commands and inspect their exit statuses:

```
uv run python work/logtool.py count data/listening.log
uv run python work/logtool.py errors data/listening.log --level WARN
uv run python work/logtool.py errors data/listening.log --level FATAL
uv run python work/logtool.py count missing.log
```

The statuses are 0, 1, 0 and 2 respectively.

> ***Hint :*** *The slides main and the Exit Status and Standard Output and Standard Error show the entry point and diagnostic stream.*

If time permits

Accept `-` as the input path and read stdin. Add `--quiet` to suppress results while retaining the exit status. Add PEP 723 metadata to make dependencies explicit if adding a rich table.

> ***Hint :*** *The demo `unix_citizen` shows stdin, quiet mode and grep-style statuses. The slide Inline Script Metadata shows a PEP 723 block.*

Exercise 3 : Checking a Music Library

Build `library_check.py`, a report of missing durations and duplicate titles. The supplied `data/tracks-import.csv` contains 41 records: one duplicated title and one missing duration. Its play counts are made up, not from work.

Start from `exercises/starters/03-modern-toolbelt/`. Run from `libraries/`. The solution is in `solutions/solution03-modern-toolbelt/`.

Step 1 : Read the import

Use `csv.DictReader` to load `data/tracks-import.csv`. Print the number of records. Keep the file path as an argument to `load(source: Path) -> list[dict[str, str]]`.

> ***Hint :*** *The demo `tqdm_basics` loads the track CSV before starting its loop.*

Step 2 : Find import problems

Write `find_issues(rows)` returning a list of `(title, problem)` tuples. Report `missing duration` when `duration_s` is empty. Report `duplicate title` when a title has already appeared. Maintain a set of seen titles during the loop. Do not discard either record of a duplicate; report what was found.

> ***Hint :*** *The demo `tqdm_write` tests an empty field while visiting rows. Sets and loops were introduced in the fundamentals course.*

Step 3 : Show progress

Wrap the checking loop with `tqdm`. Give it `desc="Checking"` and `unit="track"`. Add `time.sleep(0.01)` inside the loop to simulate slower file inspection. Use `tqdm.write` if printing a message while the bar is active.

> ***Hint :*** *The slides tqdm Progress Bars and tqdm.write show these calls.*

Step 4 : Print the report

Build a rich table with `Title` and `Problem` columns. Sort the issues by title and add one row per issue. Colour the problem cell with rich markup. Print the record and issue counts after the table. The fixture has two issues.

> ***Hint :*** *The slide rich Tables shows `Table`, `add_column`, `add_row` and cell markup.*

If time permits

Create `library_check_cli.py` with a typer command. Accept a source path and `--strict`. Print the same table, returning status 1 when strict mode finds issues and status 2 when the file cannot be read. Otherwise return status 0.

> ***Hint :*** *The slides typer and typer Trade-offs show annotated parameters and `typer.Exit`. main and the Exit Status in Command-line Programs shows the meaning of exit statuses.*

Exercise 4 : Scraping a Page Into a Database

In this exercise you build the three steps of a scraper against a real site : you fetch a page from `books.toscrape.com`, parse the twenty books out of the markup, write them into a SQLite database with parameterised inserts, and query the result back. The site exists to be scraped, so you are welcome there as long as you are polite about it.

Work in a file of your own, for example `scrape_books.py`, and run it with `uv run scrape_books.py` from the repository root. The solution is in `solutions/solution04-web-scraping/`.

The supplied solution runs offline against `data/books-fixture.html`, which contains the first page. Pass `--live` to fetch the site and include the optional second page. Without internet access, load the fixture with `Path("data/books-fixture.html").read_text(encoding="utf-8")` in Step 1 and continue with the same parsing and database steps.

Start from `exercises/starters/04-web-scraping/`. Run from `libraries/`, so relative data paths resolve.

Step 1 : Prove the fetch and the parser work

> ***Hint :*** *The slide Fetching a Page shows the operation used in this step.*

Fetch the page and make a soup from it. Always pass a `timeout`, because `requests` has no default and a server that goes quiet would hang your script for ever :

```
import requests
from bs4 import BeautifulSoup

URL =
"https://books.toscrape.com/catalogue/category/books/mystery_3/index.ht
ml"
html = requests.get(URL, timeout=10).text
soup = BeautifulSoup(html, "lxml")
```

Print two things : the page title, and how many `article` elements with class `product_pod` the page contains. You should see the word `Mystery` and the number `20`. That is the whole of step one.

If the title comes out full of newlines, tidy it with `" ".join(soup.title.text.split())`.

Step 2 : One book

> ***Hint :*** *The slide Text and Attributes shows the operation used in this step.*

Take the first `article.product_pod` and pull three fields out of it :

- the title, which is the `title` attribute of the link inside the `h3`
- the price, which is the text of the `p` with class `price_color`
- the rating, which is the second class name on the `p` with class `star-rating`

```
pod = soup.select_one("article.product_pod")
link = pod.select_one("h3 a")
print(link["title"], link["href"])
print(pod.select_one("p.price_color").text)
print(pod.select_one("p.star-rating")["class"])
```

Note that the price text carries a currency sign and the rating is a word rather than a number. Deal with both : strip everything that is not a digit or a dot out of the price and turn it into a `float`, and map the words `One` to `Five` onto the numbers 1 to 5 with a dictionary.

Step 3 : All twenty, as records

> ***Hint :*** *The slide Extracting Records shows the operation used in this step.*

Write a function that does step 2 for every book on the page and returns a list of dictionaries, one per book :

```
def parse_books(html: str, page_url: str) -> list[dict]:
    ...
```

Each record should carry `title`, `price_gbp` as a `float`, `rating` as an `int`, `availability` as text, and `url`. Build the url with `urllib.parse.urljoin` against the page address so the stored link is absolute :

```
url = urljoin(page_url, link["href"])
```

Print the first three records and the number of records. After this function returns, no part of your program should ever mention HTML again.

Step 4 : Into the database

> ***Hint :*** *The slide Parameterised Inserts shows the operation used in this step.*

Create `data/exercise-books.db` if it is not there, with one table. Give `url` a `UNIQUE` constraint, because one page is one book :

```
SCHEMA = """
CREATE TABLE IF NOT EXISTS books (
    id        INTEGER PRIMARY KEY,
    title     TEXT    NOT NULL,
    price_gbp REAL    NOT NULL,
    rating    INTEGER NOT NULL,
    url       TEXT    NOT NULL UNIQUE,
    last_seen TEXT    NOT NULL
);
"""
```

Insert the records with `executemany` and placeholders, never with a formatted string. Named placeholders take your dictionaries directly, once you have added the timestamp :

```
INSERT = "INSERT INTO books (title, price_gbp, rating, url, last_seen)
VALUES (...)"
seen_at = datetime.now(UTC).isoformat(timespec="seconds")
conn.executemany(INSERT, [book | {"seen_at": seen_at} for book in
books])
conn.commit()
```

Remember that `sqlite3` does not commit for you. Print how many rows the table holds when you are done.

Step 5 : Query it back

> ***Hint :*** *The slide SQL Injection shows the operation used in this step.*

With the data in a table, the questions are ordinary SQL. Answer three of them and print the results :

- the five most expensive books, title and price
- how many books there are at each rating, and the average price per rating

- the number of books priced under twenty

```
rows = conn.execute(
    "SELECT rating, COUNT(*), ROUND(AVG(price_gbp), 2) FROM books GROUP
BY rating"
).fetchall()
```

Setting `conn.row_factory = sqlite3.Row` before the query lets you write `row["rating"]` instead of `row[0]`, which is worth it as soon as a query has more than two columns.

Step 6 : Run it twice

> ***Hint :*** *The slide Upsert shows the operation used in this step.*

Run your script a second time. If it fails with `sqlite3.IntegrityError` on the unique url, that is the constraint doing its job.

Make the script safe to re-run by turning the insert into an upsert :

```
INSERT INTO books (...) VALUES (...)
ON CONFLICT(url) DO UPDATE SET
    price_gbp = excluded.price_gbp,
    last_seen = excluded.last_seen
```

Run it twice more and confirm the row count stays at twenty while `last_seen` moves.

Step 7 : Optional, the second page, politely

> ***Hint :*** *The slide robots.txt and Rate Limiting shows the operation used in this step.*

The Mystery category has a second page of twelve more books :

```
PAGE_URLS = [

"https://books.toscrape.com/catalogue/category/books/mystery_3/index.ht
ml",

"https://books.toscrape.com/catalogue/category/books/mystery_3/page-
2.html",
]
```

Loop over both, parse each, and upsert the lot. Three things make the difference between a scraper a site tolerates and one it blocks :

```
session = requests.Session()
session.headers["User-Agent"] = "MyScraper/1.0 (you@example.com)"
time.sleep(2)          # between requests, not before the first one
```

Use one `Session` so the connection is reused, name your program in the `User-Agent`, and sleep between the two requests. The row count should go from twenty to thirty two, and running the whole thing again should leave it at thirty two.

Exercise 5 : SQLAlchemy

In this exercise you point SQLAlchemy at the database the scraping module filled in, describe its table as a Python class, and then build a small catalogue of your own with a relationship in it. Everything you write goes into `data/exercise-library.db`, so the scraped `data/books.db` is only ever read and cannot be damaged.

Start from `exercises/starters/05-sqlalchemy/`. Run from `libraries/`, so relative data paths resolve.

Step 1 : Connect to something that already exists

> ***Hint :*** *The slide Engine and URL shows the operation used in this step.*

Make a file `library.py`. Create an engine pointing at the scraped database and ask it what is in there. This step exists to prove the environment works :

```
from pathlib import Path
from sqlalchemy import MetaData, create_engine

DATA = Path(__file__).resolve().parent / "data"
engine = create_engine(f"sqlite:///{DATA / 'books.db'}")

metadata = MetaData()
metadata.reflect(bind=engine)
print(list(metadata.tables))
```

You should see `['books']`. Now print the column names of that table, and then the number of rows in it. If `data/books.db` does not exist, run `uv run demos/04-web-scraping/12_rebuilding_from_seed.py` first, which builds it from the checked-in seed without touching the network.

Step 2 : Describe the table as a class

> ***Hint :*** *The slide Declarative Models shows the operation used in this step.*

Reflection is convenient, but it gives you tuples. Write a `Book` model that matches the table you just inspected, so that queries hand back objects instead :

```
class Base(DeclarativeBase):
    pass


class Book(Base):
    __tablename__ = "books"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    title: Mapped[str] = mapped_column(String(200))
    # add price_gbp, rating and category
```

Give it a `__repr__` so that printing one is readable. Then open a `Session` and print the first three books.

You do not need `create_all()` here; the table already exists and your class is describing it, not creating it.

Step 3 : Ask it questions

> ***Hint :*** *The slide Session shows the operation used in this step.*

Using `select()` and the session, answer each of these. Print the answer with a line saying what it is :

- Every book rated 5, ordered by price, most expensive first.
- The number of books costing more than £30.
- The average price at each rating, as a small table.

For the last one you want `func.count()` and `func.avg()`, and `group_by`. Compare what you write with the SQL you would have written by hand; they should look similar enough to be reassuring.

Step 4 : Your own database, with two tables

> ***Hint :*** *The slide ORM Queries shows the operation used in this step.*

Now build something of your own in `data/exercise-library.db`. Two models :

- `Shelf`, with an `id` and a `name`.
- `Volume`, with an `id`, a `title`, and a foreign key to `Shelf`.

Wire them together with a relationship in both directions :

```
class Shelf(Base):
    __tablename__ = "shelf"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    name: Mapped[str] = mapped_column(String(60))
    volumes: Mapped[list[Volume]] =
relationship(back_populates="shelf")
```

Call `Base.metadata.create_all(engine)` against the new database, then create two shelves with a few volumes each. Add only the shelves to the session, not the volumes, and check that the volumes are saved anyway.

Step 5 : Read it back both ways

> ***Hint :*** *The slide Relationships shows the operation used in this step.*

Print every shelf with the titles on it, using the relationship attribute. Then print the same information again as a single query with an explicit `join`.

Both should produce the same lines. Satisfy yourself about which one issued how many queries; passing `echo=True` to `create_engine` will show you every statement as it is sent, which is the quickest way to see the difference.

Step 6 : Change something

> ***Hint :*** *The slide Relationships shows the operation used in this step.*

Move one volume from one shelf to the other by assigning to its `shelf` attribute, then commit. Print both shelves again to confirm it moved.

Notice that you did not write an `UPDATE` statement, and that you did not touch the foreign key column yourself.

Step 7 : Optional, hand it to pandas

Load your `volume` table into a DataFrame with `pandas.read_sql`, passing the engine rather than a connection. Group it by shelf and count the rows.

Then do the same aggregation in SQL instead, and compare. Which one you would reach for in real work depends on how big the table is and on what you want to do next, and it is worth having an opinion about both.

Exercise 6 : Listening Habits with NumPy

Analyse a fictional week of listening. `data/plays-hourly.csv` has one row per day and one numeric column per hour. Counts are made up, not from work or a streaming account. A high-count hour is called a "binge hour" for this report; the label is a teaching category, not a health judgement.

Run commands from `libraries/`. Create your scripts in `work/numpy_listening/`. The complete solution is in `solutions/solution06-numpy/`. Each script reloads the CSV so it can run independently.

Step 1 : Load the grid

Create `01_load.py`. Read the numeric columns, excluding the header and day labels:

```
from pathlib import Path

import numpy as np

DATA = Path("data")
plays = np.loadtxt(DATA / "plays-hourly.csv", delimiter=",",
skiprows=1,
                   usecols=range(1, 25), dtype=np.int64)
```

Print the shape, dtype, size and `plays[:2, :6]`. The shape is `(7, 24)` and the array contains 168 counts.

> ***Hint :*** *The slide Reading a CSV and demo* `csv_into_array` *show* `loadtxt` *and* `usecols`.

Step 2 : Summarise each day

Create `02_per_day.py`. Compute each day's mean, minimum and maximum. Print the results beside `Mon`, `Tue`, `Wed`, `Thu`, `Fri`, `Sat` and `Sun`. Find the day with the highest mean using `np.argmax`.

> ***Hint :*** *The slide axis Argument shows that removing the hours with* `axis=1` *leaves one result per day. Aggregations shows* `argmax`.

Step 3 : Summarise each hour

Create `03_per_hour.py`. Compute the mean count per hour across the week. Print the busiest and quietest hour with their means. Print each hour's mean beside a bar of `#` characters. Use a loop to format the results; compute the means with an array operation.

> ***Hint :*** *The slide axis Argument shows that* `axis=0` *removes days and leaves hours. The result has shape* `(24,)`.

Step 4 : Find binge hours

Create `04_alerts.py`. Build a mask for counts above 10. Print the number of matching cells, then print each matching day, hour and count. Also print which days have at least one such hour and count the cells above 6 and at most 10.

> ***Hint :*** *The slide Boolean Masking shows masks and* `nonzero`. *axis Argument explains how an* `any(axis=1)` *reduction gives one boolean per day.*

Step 5 : Compare with the baseline

Create `05_baseline.py`. Calculate the median for each hour across the week, then subtract those medians from the grid. Find the day and hour with the largest positive deviation. Print its deviation.

Make an independent copy of the counts and cap every value above 10 at 10. Print the maximum of both arrays to prove that the original was preserved.

> ***Hint :*** *The slides Broadcasting and Aggregations show compatible shapes, median and* `unravel_index` *. Views and Copies shows an independent copy.*

If time permits

Create `06_bands.py` . Label counts above 10 as `binge` , counts above 6 as `regular` , and the rest as `quiet` . Count each label overall and count binge hours per day. Save the 24 hourly means with `savetxt` .

> ***Hint :*** *The slide np.where shows nested conditions. Reading a CSV shows* `savetxt` *.*

Exercise 7 : Pandas

In this exercise you take the book catalogue that the scraping module put into SQLite, pull it into pandas, and turn it into the spreadsheet somebody upstairs actually asked for. You will read a database with `read_sql`, filter it, add a column, summarise it with `groupby`, and write the answer out as an Excel workbook with two sheets. The report uses the same selection, grouping and export operations as the track demos.

Start from `exercises/starters/07-pandas/`. Work from `libraries/`, with your own scripts in `work/book_report/`. The solution files are in `solutions/solution07-pandas/`. Keep one script per step. Each step reloads the data at the top, which costs nothing on twenty rows and means any file runs on its own.

> ***If*** `data/books.db` ***is not there yet****, the scraping module has not been run on this machine. Build it with* `uv run demos/04-web-scraping/12_rebuilding_from_seed.py`*, which recreates the table from the checked-in* `data/books-seed.sql` *without scraping anything.*

Step 1 : Open the database

> ***Hint :*** *read_sql and demo* `read_sql` *show a query result becoming a frame.*

Create `01_load.py`.

Connect to the database, ask it what tables it has, and read one of them into a DataFrame :

```
import sqlite3
from pathlib import Path

import pandas as pd

DATA = Path("data")

conn = sqlite3.connect(DATA / "books.db")
print(pd.read_sql("SELECT name FROM sqlite_master WHERE type =
'table'", conn))
books = pd.read_sql("SELECT * FROM books", conn)
conn.close()

print(books.shape)
print(books.head())
```

You should see the `books` table and a shape of `(20, 9)` when using the supplied seed.

Step 2 : Look at what you loaded

> ***Hint :*** *info and describe and Missing Values show the inspection methods.*

Create `02_inspect.py` and read the table again.

Print `books.info()` and `books.describe()`. Between them they answer the three questions worth asking about the catalogue : how many rows are there, what type is each column, and is anything missing.

Then answer these questions :

1. How many missing values are there in each column? `isna()` gives a mask and a mask sums to a count.
2. What distinct values does the `category` column hold, and how many books are in each? `value_counts()` does both at once.

Note the dtype of `price_gbp` and the dtype of `rating`. One is a float and one is an integer, and pandas worked that out from the database rather than being told.

Step 3 : Filter down to the rows that matter

> ***Hint :*** *Combining Filters shows masks,* `isin` *and* `between`*.*

Create `03_filter.py`.

Build the filters as named variables first, then combine them. A mask with a name is a mask you can read six weeks later :

```
well_rated = books["rating"] >= 4
in_stock = books["availability"] == "In stock"
affordable = books["price_gbp"] < 40.0
```

Print how many rows each single filter matches, then print the rows that satisfy all three. Remember `&` rather than `and`, and remember the parentheses around each comparison when you write the combination in one line.

Then do the same thing two other ways, using the same masks :

1. `books[books["rating"].isin([4, 5])]`, which replaces a chain of ORs.
2. `books[books["price_gbp"].between(20.0, 40.0)]`, which replaces two comparisons and includes both ends.

Step 4 : Add the columns the report needs

> ***Hint :*** *Deriving a Column and demo* `np_where` *show column calculations.*

Create `04_derive.py`.

The catalogue is priced in pounds and the report wants euros. Add a column, rounded to two decimals :

```
books["price_eur"] = (books["price_gbp"] * RATE).round(2)
```

Use the fixed teaching rate `RATE = 1.17` ; it is an example value, not a current exchange rate.

Then add a `band` column that says `budget` under 20 euros, `mid` under 45, and `premium` above that. `np.where` nested inside another `np.where` does it in one expression, as shown in the NumPy demo `np_where` .

Finally sort the whole thing by price, most expensive first, and print the top five titles with their two prices. `sort_values("price_eur", ascending=False).head(5)` is the whole of it, and `nlargest(5, "price_eur")` says the same thing in fewer words.

Step 5 : Summarise by rating

> ***Hint :*** *Named Aggregation shows named output columns and* `reset_index` .

Create `05_group.py` .

This is the `GROUP BY` of the exercise. Produce one row per star rating, with four columns : how many books have that rating, the mean price, the cheapest and the dearest.

Use named aggregation so the output columns arrive with the names you want rather than names you have to fix afterwards :

```
per_rating = (
    books.groupby("rating")
    .agg(
        titles=("title", "count"),
        mean_price=("price_eur", "mean"),
        ...
    )
    .round(2)
    .reset_index()
)
```

Print it. Then answer the question it raises : does a higher rating cost more? Sorting the result by `mean_price` is enough to see.

Step 6 : Export to Excel

> ***Hint :*** *Writing Files shows* `ExcelWriter` *and* `index=False` *.*

Create `06_export.py` .

Build both frames again : the filtered list of well-rated books from step 3 with the derived columns from step 4, and the per-rating summary from step 5. Then put them in one workbook, one sheet each :

```
with pd.ExcelWriter(DATA / "book-report.xlsx", engine="openpyxl") as
workbook:
    shortlist.to_excel(workbook, sheet_name="shortlist", index=False)
    per_rating.to_excel(workbook, sheet_name="by rating", index=False)
```

Pass `index=False` on both, because the row numbers are pandas' business and not the reader's. Write a CSV of the shortlist as well with `to_csv` , for a program that reads CSV.

Print the path and the sheet names at the end, then open the file and check that it is really a spreadsheet.

Optional : a chart and a second table

Create `07_chart.py` if you have time.

Two things, either order.

A picture. Plot the mean price per rating as a bar chart. Select the non-interactive backend before importing `pyplot`, so the script writes a file instead of waiting for you to close a window :

```
import matplotlib

matplotlib.use("Agg")
```

Then `per_rating.plot(kind="bar", x="rating", y="mean_price")`, take `.get_figure()` from the axes it returns, and `savefig` it into `data/`.

A join. Build a small lookup frame giving each star rating a word :

```
words = pd.DataFrame({"rating": [1, 2, 3, 4, 5], "verdict": ["poor",
...]})
```

`merge` it onto the summary on `rating` and print the result, so the report reads in English rather than in integers. Then try the same merge with `how="left"` after deleting one row from the lookup, and use `indicator=True` to see exactly which rows failed to match.

Exercise 9 : Running Other Programs

APPENDIX

A small appendix exercise, and the one most worth doing on your own Linux machine rather than in the classroom, because there the commands are real and the shell is a POSIX one. You will build a short report about a directory of files by asking other programs for the answers, and then replace half of it with Python that needs no other program at all.

Start from `exercises/starters/09-appendix-subprocess/`. Run from `libraries/`.

Step 1 : Ask one program one question

> ***Hint :*** *The slide subprocess.run shows the operation used in this step.*

Make a file `report.py`. Run `git --version` and print the version it reports, without the surrounding text :

```
import subprocess

finished = subprocess.run(
    ["git", "--version"], capture_output=True, text=True, check=True
)
print(finished.stdout.strip())
```

Change `check=True` to `check=False` and run it against a command that does not exist, such as `["git", "nonsense"]`. Look at `returncode` and at `stderr`.

Step 2 : Read an exit code that means something

> ***Hint :*** *The slide Exit Codes shows the operation used in this step.*

Run `grep -c ERROR` against `data/listening.log` and print the count.

Then run it against a pattern that matches nothing. You will get a return code of 1 and a count of zero, and that is not a failure; it is `grep` telling you it found nothing.

Write a function that returns the number of matching lines, treating a return code of 0 or 1 as success and anything else as a real error :

```
def count_matches(pattern: str, path: Path) -> int:
    found = subprocess.run(...)
    if found.returncode > 1:
        raise RuntimeError(f"grep failed: {found.stderr.strip()}")
    ...
```

Step 3 : Make it fail safely

> ***Hint :*** *The slide Missing Programs shows the operation used in this step.*

Add a check at the top of the script that `grep` is actually installed, using `shutil.which`. If it is missing, print a clear message to `stderr` and exit with a non-zero status, using what you built in block 02.

Test it by asking for a program you know is not there.

Step 4 : Two commands, no shell

> ***Hint :*** *The slide Standard Input and Pipelines shows the operation used in this step.*

Produce a count of each log level in `data/listening.log`, the way you would with `grep -o -E 'INFO|WARN|ERROR' file | sort | uniq -c`, but as separate `subprocess.run` calls passing text between them with `input=`.

Print the result as a small table.

Step 5 : Give it a timeout

> ***Hint :*** *The slide Timeouts shows the operation used in this step.*

Add `timeout=` to every call. Pick something generous, five seconds say, so it never fires in normal use.

Then prove it works by running a command that sleeps for longer than that, and catching `TimeoutExpired`. On Linux, `["sleep", "30"]` will do.

Step 6 : Now throw half of it away

> ***Hint :*** *The slide pathlib Alternatives shows the operation used in this step.*

Rewrite Step 4 in pure Python: read the file with `Path.read_text`, and count the levels with a dictionary or `collections.Counter`. No `subprocess` at all.

Compare the two versions. Time them if you like; the difference will be larger than you expect, because the first version starts three processes and the second starts none.

Which one you would rather maintain is the real question, and the answer is not the same for every script.

Step 7 : Optional, something genuinely useful

Point the script at a directory of your own and have it report something you actually want to know, using a program that has no Python equivalent worth writing. Some possibilities :

- `git log --format=%an` piped into a count, for who has touched a repository.
- `systemctl is-active` for a list of services, reporting only the ones that are not running.
- `df -P` parsed into a warning for any filesystem over a threshold you choose.

Keep the rule from the last slide in mind : let the other program do what only it can do, and do the rest in Python.